Hochat Online

Final Project Presentation Guide · Group 2

WMA4 – Advanced Web Development · Tarlac State University, College of Computer Studies
A Web-Based Food Ordering System for Hochat Café · Laravel 13 · Inertia + React · MySQL

1. Presentation Time Breakdown

Time	Part	What to prepare (see section)
7 minutes	Actual System Demonstration	Demo script for all 3 roles + API (§4)
8 minutes	Backend Explanation and Debugging	Architecture, code walkthrough, security, database (§5)
10 minutes	Live Debugging	Debugging method, tools, common errors (§6)
11 minutes	Questions and Answers	Likely questions with answers (§7)
36 minutes	**Total**	

2. Evaluation Criteria and How We Meet Them

Criterion	Points	Evidence in our system
1. System Functionality and Completeness – working features, CRUD operations, system processes, alignment with the approved scope	30	Register/login, password reset, profile; menu by category + search; cart; checkout (Dine-In/Delivery, ₱50 fee, Gerona only, pre-orders); order history; 1-minute cancel; staff menu CRUD with photos; order status updates and cancel; Order Status Report; REST API.
2. Backend Implementation and Code Quality – server-side logic, OOP, code organization, authentication, authorization, security, API	30	Laravel MVC; Eloquent models with relationships; Form Request classes for validation; `OrderPolicy` + role middleware; `Role` enum; shared `Order::placeFor()`; Sanctum token API with JSON Resources; 64 Pest tests; Pint-formatted code.
3. Database Design and Management – relationships, data integrity, SQL operations, proper data handling	15	5 related tables with foreign keys; transactions when placing orders; items used in past orders cannot be deleted; enums for order type/status; decimal money columns; migrations + seeders.
4. Technical Explanation and Individual Understanding – architecture, backend processes, implementation, individual contributions	15	Each member explains their own part (§3). Use the request flow diagram (§5.1).
5. Revised Manuscript and Source Code Submission – updated documentation, complete ZIP source code, importable SQL database	10	Revised paper (instructor's 4 revisions), README, ZIP of source, `.sql` export. Commands in §8.
	Total	**100**

3. Suggested Roles During the Presentation

Member	Paper role	Presents / answers about
Tan, Sean Jedrek P.	Leader / Documentation	Opening, scope, timekeeping, revised manuscript, Q&A host
Bucud, Ralph Antoine S.	Front-End Developer	Demo: customer side (menu, cart, checkout, receipt); React pages
Rombaoa, Jayrald C.	UI/UX Designer	Mockups vs. final screens, layout, theme, responsiveness
Lanuza, John Mathew C.	Back-End (Customer)	Auth, password reset, profile, cart, `Order::placeFor()`, cancel policy
Mercado, Regin Ray T.	Back-End (Admin/Staff)	Roles/middleware, menu CRUD, order management, report, API
Tayag, Harvey R.	Database / QA	Tables and relationships, SQL export, Pest tests, leads live debugging
Manansala, Kian Wayne Q.	System Analyst	Architecture, use case/DFD vs. the built system, order-status flow

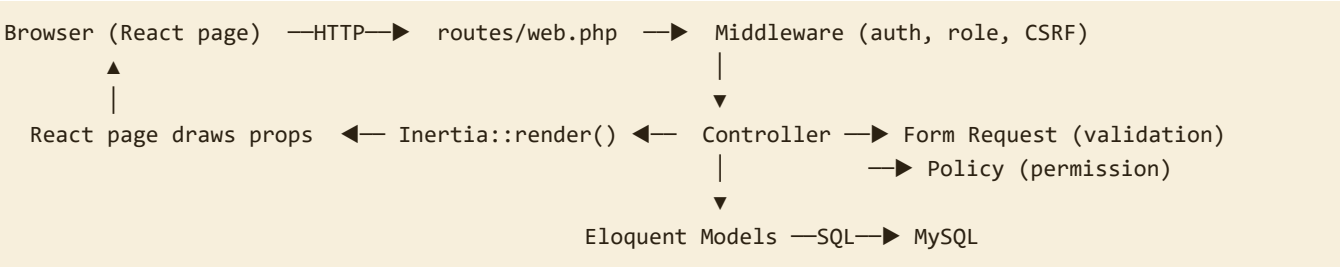
4. Part 1 – System Demonstration (7 minutes)

> **Accounts** (password for all: `password`): Admin `admin@hochat.test` · Staff `staff@hochat.test` · Customer `user@hochat.test`. Use two browser windows (one normal, one private) so Customer and Staff are logged in at the same time.

Time	Who	Show
0:00–0:30	Guest	Home page (slideshow), About Us, Contact Us. Register page.
0:30–2:30	Customer	Log in → Menu: category tabs, search, sold-out item → Add to cart → Cart: +/−, remove → choose **Delivery** (₱50 fee appears) → if address is outside Gerona show the error → Profile: fix address → Proceed to Checkout → Receipt.
2:30–4:30	Staff	Report page (counts by status) → Manage Orders: the new order appears **without reloading** → Mark as Preparing → Ready → Completed. Customer's receipt updates by itself. Cancel another order.
4:30–5:30	Customer	Place a new order and cancel it within 1 minute. Show that an older order has no Cancel button.
5:30–6:15	Admin	Manage Menu: add item with photo, mark sold out, delete an unused item; deleting an item in past orders is refused. Staff has no Delete button.
6:15–7:00	Postman	`POST /api/v1/login` → token → `POST /cart` → `POST /cart/checkout` (201) → staff token `PATCH /admin/orders/{id}/status`.

5. Part 2 – Backend Explanation (8 minutes)

5.1 How a request travels (architecture)



The API (`routes/api.php`) follows the same path but returns JSON instead of a React page, and logs users in with Sanctum **tokens** instead of sessions.

5.2 Key files to show

File	What to say
`routes/web.php`, `routes/api.php`	Every URL and the middleware protecting it (`auth`, `role:Staff,Administrator`, `role:Administrator`, `auth:sanctum`).

`app/Models/Order.php` → `placeFor()`	One method places orders for both website and API: empty cart, delivery address, Gerona, sold-out checks, then a **database transaction** creates the order and its items.
`app/Policies/OrderPolicy.php`	Customer may view only their own order; may cancel only if Pending and ≤ 60 seconds old.
`app/Http/Middleware/EnsureUserHasRole.php`	Role-based access: returns 403 when the role is not allowed.
`app/Http/Requests/*`	Validation rules in their own classes (OOP, single responsibility).
`app/Http/Controllers/Admin/DashboardController.php`	Report: `count(*) GROUP BY status` for a date range.
`app/Http/Controllers/Api/V1/*` + `app/Http/Resources/*`	REST endpoints; Resources choose which fields go into the JSON.

5.3 OOP and security points

OOP

- Models are classes with relationships (`hasMany`, `belongsTo`).
- Encapsulation: business rules live in `Order::placeFor()`, not in pages.
- `Role` is a PHP **enum**.
- Controllers/Requests extend Laravel base classes (inheritance).

Security

- Passwords hashed (bcrypt); never stored as text.
- Eloquent uses prepared statements → no SQL injection.
- CSRF tokens on every form; session regenerated on login, destroyed on logout.
- Server-side role checks + policy; login throttled (5 tries/min).

5.4 Database

```
users ──< orders ──< order_items >── menu_items >── categories
```

orders: order_type (Dine-In/Delivery), status (Pending/Preparing/Ready/Completed/Cancelled), delivery_fee, total_amount = subtotal + delivery_fee, scheduled_time (pre-orders). **order_items** stores sub_total at the time of ordering, so later price changes do not change old receipts.

6. Part 3 – Live Debugging (10 minutes)

6.1 Method: follow the request

1. **Reproduce** the problem and open the browser DevTools (**F12** → **Network**). Note the URL and **status code**.
2. **Find the code**: `php artisan route:list --path=cart` shows the controller method.
3. **Read the error**: newest entry at the bottom of `storage/logs/laravel.log`.
4. **Inspect values**: `dd($variable)` or `logger('label', [$x])` in PHP; `console.log()` in React.
5. **Fix**, then prove it: run the related test `php artisan test --filter=cancel`.

6.2 Status code → where to look

Symptom	Meaning	Look at
500	PHP error	`storage/logs/laravel.log`
403	Not allowed	Route `role:` middleware or `OrderPolicy`
404	Wrong URL / record missing	`php artisan route:list`, the ID
419	Page expired (CSRF)	Reload the page
422 / red text	Validation failed	`rules()` in `app/Http/Requests`
Redirect to /login	Not logged in	`auth` middleware
401 (API)	Missing/invalid token	Postman *Authorization* → *Bearer*

6.3 Errors we already met (and the fix)

Error	Fix
`Unable to locate file in Vite manifest`	Vite not running or build outdated: `composer run dev` or `npm run build`
500 on every page / connection refused	MySQL stopped: start it in Laragon
`Cannot redeclare class …`	Code pasted into the wrong file; `namespace` must match the folder
Page not found: `Menu/Index`	Page name must match the file exactly (`menu/index`)
`.form is not a function`	`php artisan wayfinder:generate --with-form`
Status `null` in API response	Set `'status' => 'Pending'` when creating (found by a test)
GitHub check fails (formatting)	`npx vp check --fix`

6.4 Practice scenarios (rehearse these)

- **"Staff can delete menu items."** → check `routes/web.php`: is `destroy` inside the `role:Administrator` group?
- **"Customer can cancel after 1 minute."** → `OrderPolicy::cancel()`, the `<= 60` check; run `--filter=cancel`.
- **"Delivery fee missing."** → `Order::deliveryFeeFor()` and `placeFor()`; test `--filter="delivery fee"`.
- **"Cart count wrong."** → `HandleInertiaRequests::share()` → `cartCount`.

- **"Price shows wrong total."** → `sub_total` calculation in `placeFor()`; check the row with `php artisan tinker`.

> **Tip:** write a failing test that reproduces the bug first, fix the code, then show the test passing. Run all tests with `php artisan test` (64 tests).

7. Part 4 – Questions and Answers (11 minutes)

Likely question	Short answer
Why Laravel and not plain PHP?	MVC structure, Eloquent ORM (prepared statements), built-in auth, validation, CSRF, migrations and testing; the proposal planned the Laravel migration for the Final Term.
How do you stop SQL injection?	All queries use Eloquent / query builder, which bind values with prepared statements.
How are passwords stored?	Hashed with bcrypt through Laravel's `hashed` cast; checked with `Auth::attempt()` / `Hash::check()`.
How does role-based access work?	`role:` middleware on route groups (403 if not allowed) plus `OrderPolicy` for per-order rules; the React UI only hides buttons.
How is the 1-minute cancel enforced?	`OrderPolicy::cancel()`: own order, status Pending, `created_at` ≤ 60 s ago. Checked on the server, also in the API.
What if two steps of ordering fail halfway?	`DB::transaction()`: the order and its items are saved together or not at all.
Where is the cart stored?	Website: in the session. API: in the cache per user (tokens have no session).
Why can't some menu items be deleted?	They appear in past orders; deleting would break receipts and foreign keys, so we mark them sold out instead.
Is it real-time?	Pages poll the server every 10 s (report 30 s) with Inertia `usePoll`; no WebSockets.
Why is there no Pickup?	Out of scope per the revised paper; only Dine-In and Delivery. Payment: Cash on Delivery / Pay at Counter.
How is delivery limited to Gerona?	Checkout requires the profile address to contain "Gerona"; a simple rule, noted in Limitations.
How did you test the system?	64 automated Pest feature tests on a temporary SQLite database, plus manual testing of all 3 roles.
Where is it hosted?	*Fill in your group's final choice.* Developed locally on Laragon.
What was your individual contribution?	*Each member: name the files and features you built (§3).*

8. Submission Checklist (Criterion 5)

Importable SQL database

```
mysqldump -u root hochat_online > hochat_online.sql
```

Test it: create an empty database, import the file in HeidiSQL/phpMyAdmin, and open the site.

Complete ZIP of the source code

```
git archive -o Hochat-Online-source.zip HEAD
```

This packs every committed file (no `vendor`, `node_modules` or `.env` secrets). The README explains how to install it.

Revised manuscript

- Roles consistent (Staff + Administrator in Features); remove "oversee/remove users".
- Hosting/deployment environment named in Tech Stack and Architecture.
- Figure 9 replaced with a screenshot of the Order Status Report.
- Pickup removed everywhere; payment = Cash on Delivery / Pay at Counter.
- Data dictionary updated (real column names, `delivery_fee`); tech stack updated; Success indicator 3 fixed.

9. Before You Present

	Task
☐	Start **MySQL in Laragon**, then run `composer run dev` (Laravel + Vite).
☐	Fresh data: `php artisan migrate:fresh --seed` (deletes all data!) then add real menu items and photos.
☐	Run `php artisan test` — all 64 should pass. If pages fail with "Vite manifest", run `npm run build`.
☐	Place a few orders in different statuses so the report is not empty.
☐	Postman collection saved: login, menu, cart, checkout, orders, cancel, status update.
☐	Open the reset-email log once: `storage/logs/laravel.log`.
☐	Rehearse with a timer: 7 / 8 / 10 / 11 minutes.
☐	Every member has committed to GitHub from their own account.